Git and Github

About Author:

Name :Balareddy Gari Manasa
Domain: Full Stack Web Development
Trainee: NextGen Freedu
Batch :2
Date :6-02-2026
Mentor: Sravani Mam

1.Version Control system

A **Version Control System (VCS)** is a system that:

- Tracks changes in files over time
- Stores history of every modification
- Allows reverting to any previous version
- Enables multiple people to work together safely

<u>Types of version control system</u>

1. Local Version Control System

📍 Stored only on **one computer**

How it works:

- Copies of files are stored locally
- Manual tracking

Disadvantages:

- No collaboration
- No backup
- Easy to lose data

2. Centralized Version Control System (CVCS)

📍 One **central server**

Examples:

- SVN(Apache subverion)
- CVS

How it works:

Developer → Central Server ← Developer

Problems:

- If server fails → everything lost
- Requires constant internet
- Slower collaboration

3. Distributed Version Control System (DVCS)

📍 Every developer has a **full copy** of the project

Example:

- **Git**

How it works:

Each developer = full repository + full history

✅ Advantages:

- Works offline
- No single point of failure
- Very fast
- Best for large teams

2.What is Git

🔹 **Simple definition**

Git is a version control system.

That means:

- It **tracks changes** in your code
- It **stores history** of your project

- It lets you **go back to older versions**
- It helps **multiple people work together safely**

Git works on your local computer (laptop).

Created by **Linus Torvalds (2005)** for Linux kernel development.

What does Git actually track?

Git tracks:

- Which file changed
- What line changed
- Who changed it
- When it changed

Key Git Concepts

1.Repository (Repo)

A **repository** is:

- A folder
- That Git is watching

Once a folder becomes a repo, Git tracks everything inside it.

2.Git States

Every file in Git lives in **one of these states**:

1️⃣ **Working Directory**
→ You are editing files

2️⃣ **Staging Area**
→ You tell Git: "I want to save THESE changes"

3️⃣ **Committed**
→ Changes are permanently saved in history

Example:

```
git add file.py     # put changes in staging
```

git commit -m "Added login feature" **# save permanently**

3.What is a Commit?

A **commit** is:

- A snapshot of your project at a point in time
- With a message explaining **what you did**

4.Git Branches

What is a branch?

A **branch** is like a parallel version of your project.

- `main` → stable code
- `feature-login` → new feature

You can experiment without breaking the main code.

5.Git Data Integrity (SHA-1 Hashing)

SHA-1 (Secure Hash Algorithm-1) is a cryptographic hash function that:

- Takes **any input** (file, text, commit data)
- Produces a **fixed-length output**
- Output size = **160 bits (40 hexadecimal characters)**

Every Git object has a hash like:

`e83c5163316f89bfbde7d9ab23ca2e25604af290`

This hash is generated from:

- File content
- Metadata

No matter:

- 1 letter input
- 1 GB file

👉 Output length is **always the same**

SHA-1 Properties

🔹 **1. Deterministic**

Same input → same hash
Always.

🔹 **2. Fixed Length**

Any input size → 40-character hash

🔹 **3. Avalanche Effect**

Tiny change → completely different hash

How Commits Are Linked (Blockchain-like)

Each commit stores:

- Its own hash
- Parent commit hash

=> Git is moving to SHA-256 (newer versions)

Git Commands Related to Hashing

```
git hash-object file.txt #Shows SHA-1 hash of file content

git cat-file -p <hash>   #Displays content of Git object
```

Why this matters:

- If file changes → hash changes
- Tampering is detected
- History cannot be altered silently

<u>3 Understanding Git Configuration Levels</u>

Git has **3 levels** of config:

Level	Scope
System	Entire computer
Global	Current user
Local	Current repository

📌 Local overrides global.

Creating Your First Git Repository

Step 1: Create a Project Folder

```
mkdir my-first-git-project
cd my-first-git-project
```

Step 2: Initialize Git

```
git init
```

Output:

```
Initialized empty Git repository
```

📌 This creates a hidden `.git` folder.

What is the `.git` Folder?

This folder is the **heart of Git**.

It stores:

- Commit history
- Branches
- Hash objects
- Configuration

📌 Deleting `.git` = deleting Git history.

Checking Repository Status

```
git status
```

Shows:

- Current branch
- Untracked files
- Staged files
- Modified files

Creating & Tracking a File

Step 1: Create a File

```
touch hello.txt
```

Add content:

```
Hello Git
```

Step 2: Check Status

```
git status
```

You'll see:

```
Untracked files: hello.txt
```

Adding File to Staging Area

```
git add hello.txt
```

📌 Git now tracks this file.

First Commit (VERY IMPORTANT)

```
git commit -m "Initial commit"
```

What happens:

- Snapshot created
- SHA-1 hash generated
- Commit stored in history

Viewing Commit History

```
git log
```

Shows:

- Commit hash
- Author
- Date
- Message

Modifying a File (Lifecycle Demo)

Edit `hello.txt`:

```
Hello Git and GitHub
```

Check status:

```
git status
```

You'll see:

```
modified: hello.txt
```

Stage & Commit Again

```
git add hello.txt
git commit -m "Update greeting message"
```

📌 Now Git has **two snapshots**.

2.What is GitHub

Simple definition

GitHub is a cloud platform that hosts Git repositories.

📌 Git = tool
📌 GitHub = website/service

<u>Why do we need GitHub if we have Git?</u>

Git alone:

- Works only on your computer
- No backup
- No collaboration

GitHub gives:

i. Online backup
ii. Team collaboration
iii. Share code
iv. Show your projects to recruiters
v. Issues, pull requests, reviews

Local vs Remote Repository

Local Repo (Git)	Remote Repo (GitHub)
On your laptop	On GitHub
You commit here	You push here
Fast	Online

Basic Git commands

```
git init          # start git in a folder
git status        # check file status
git add .         # stage all files
git commit -m ""  # save snapshot
git branch        # see branches
git checkout -b x # create new branch
git merge         # merge branches
git clone URL     # download repo
git pull          # update code
git push          # upload code
```

What Problems GitHub Solves

Problem 1: Collaboration

Multiple developers:

- Work on same project
- From different locations

GitHub:

- Syncs work
- Prevents overwrites
- Manages conflicts

Problem 2: Backup

Laptop crashes
GitHub:

- Keeps your code safe in cloud

Problem 3: Code Review

Before merging code:

- Others review
- Comment
- Approve

Done via **Pull Requests**

Problem 4: Portfolio & Hiring

Recruiters:

- Check GitHub
- See projects
- See commit history
- See consistency

For students → **GitHub = resume**

WHAT GITHUB ACTUALLY HOSTS

We'll split this into two layers:

1. **Git-based objects** (core)
2. **GitHub platform features** (extended)

Git-based objects

These come directly from Git.

1.Repositories

What is a Repository?

A **repository (repo)** is:

- A container for a project
- Holds **entire Git history**
- Central place for code + metadata

What a repo contains:

- Files & folders
- Commit history
- Branches
- Tags
- Configuration

Types:

- **Public repo** → visible to everyone
- **Private repo** → restricted access

GitHub hosts the **remote copy** of your Git repo.

2.Branches

What is a Branch?

A **branch** is:

- A pointer to a commit
- Represents a line of development

Example:

```
main        → stable code
feature-ai → experimental code
```

Why GitHub hosts branches:

- Multiple developers work in parallel
- Each feature has its own branch
- Safe collaboration

On GitHub you can:

- View branches
- Compare branches
- Protect branches

3.Commits

What is a Commit?

A **commit** is:

- A snapshot of your project
- Identified by a **SHA hash**
- Has author, date, message

GitHub hosts:

- Commit history
- Visual diff (line-by-line changes)
- Author attribution

4.Tags

What is a Tag?

A **tag** is:

- A named pointer to a commit
- Used to mark **important versions**

Why tags exist:

- Mark releases
- Identify stable versions
- Roll back easily

GitHub displays tags clearly and links them to releases.

5.Releases

What is a Release?

A **release** is:

- A packaged, published version of your software
- Built on top of a **tag**

A release can include:

- Release notes
- Changelog
- Binary files (.exe, .apk, .zip)

GITHUB PLATFORM FEATURES

1.Issues

What are Issues?

Issues are used to:

- Report bugs 🐞
- Request features ✨
- Track tasks 📋
- Discuss ideas

Each issue can have:

- Title & description
- Labels (bug, enhancement)
- Assignees
- Comments

2.Pull Requests (PRs)

What is a Pull Request?

A **Pull Request** is:

> "Please review my changes and merge them."

Used when:

- Merging branches
- Collaborating in teams
- Contributing to open source

PR includes:

- Code changes (diff)
- Discussion
- Review comments
- Approval status

Flow:

```
Branch → Commit → Push → Pull Request → Review → Merge
```

3.GitHub Actions

What are GitHub Actions?

GitHub Actions provide:

- Automation
- CI/CD (Continuous Integration / Deployment)

They can:

- Run tests
- Build code
- Deploy apps
- Check formatting

Triggered by:

- Push
- Pull request
- Schedule

4.Wikis

What is a Wiki?

A **Wiki** is:

- Documentation space
- Separate from code

Used for:

- Project explanation
- Architecture diagrams
- Setup guides
- Internal docs

Good for:

- Big projects
- Team knowledge sharing

5.GitHub Pages

What is GitHub Pages?

GitHub Pages:

- Hosts static websites
- Directly from a repo

Used for:

- Portfolios
- Project demos
- Documentation sites

Thank you